

УДК 004.94

# МНОГОУРОВНЕВЫЙ СТАТИЧЕСКИЙ АНАЛИЗ ИСХОДНОГО КОДА ПРОГРАММ ДЛЯ ОБЕСПЕЧЕНИЯ КАЧЕСТВА ПРОГРАММ

© 2017 г. А.А. Белеванцев

*Институт системного программирования РАН**109004 Москва, ул. А. Солженицына, д. 25**Московский государственный университет имени М.В. Ломоносова**119991 Москва, Ленинские горы, д. 1. д. 7**E-mail: abel@ispras.ru*

Поступила в редакцию 15.07.2017

В настоящей статье обсуждаются методы анализа программ для поиска ошибок в исходном коде, которые вместе формируют многоуровневую систему анализа. Первый уровень анализа заключается в проверках по абстрактному синтаксическому дереву и внутрипроцедурном анализе потока данных, а также строит модель памяти программы для последующих уровней, для которой оцениваются значения целочисленных выражений и множества потенциальных целей указателей. Второй уровень — это межпроцедурный анализ на основе аннотаций функции, в котором интересующие свойства программы формулируются как свойства классов значений, выделенных в программе. Наконец, третий уровень анализа — это чувствительный к путям анализ, строящий формулы достижимости точек программы и отслеживающий предикаты, при которых выполняются нужные свойства, а ошибки ищутся с помощью проверки на совместность указанных формул SMT-решателем. Все упомянутые уровни анализа реализованы в инструменте Svace, который демонстрирует масштабируемость анализа до миллионов строк кода и точность в 60–90% истинных срабатываний.

## 1. ВВЕДЕНИЕ

Требования к статическому анализатору, ищущему ошибки в программе, для его использования в промышленном цикле разработки ширятся с каждым годом. В настоящее время общепринятым является ряд функциональных требований (поддержка популярных языков и классов дефектов, расширяемость) и нефункциональных характеристик (масштабируемость до проведения анализа миллионов строк кода за несколько часов, не менее 60% истинных срабатываний, пропуск незначительного количества дефектов). Анализ известных классов дефектов, рассматриваемых в общедоступных базах данных CWE, рекомендаций стандартов CERT Coding Standards, методических документов ФСТЭК РФ показывает, что обойтись единственным уровнем анализа («движком») для поиска всех этих дефектов невозможно.

С одной стороны, наиболее критические дефекты требуют для своего обнаружения межпроцедурного анализа, который поддерживает чувствительность к контексту и к путям выполнения. С другой стороны, большинство дефектов, связанных с нарушениями правил безопасного кодирования, отраслевых стандартов типа MISRA, рекомендованных языковых практик кодирования, требуют анализа уровня абстрактного синтаксического дерева (т.н. AST-уровня) и максимально детальной информации об исходном коде. На этом же уровне чаще всего требуется искать новые дефекты. Наконец, существует некоторое количество дефектов, занимающих промежуточное положение между этими двумя категориями — для них требуется межпроцедурный анализ, чувствительный к контексту выполнения, но, как правило, не требуется чувствительность к путям.

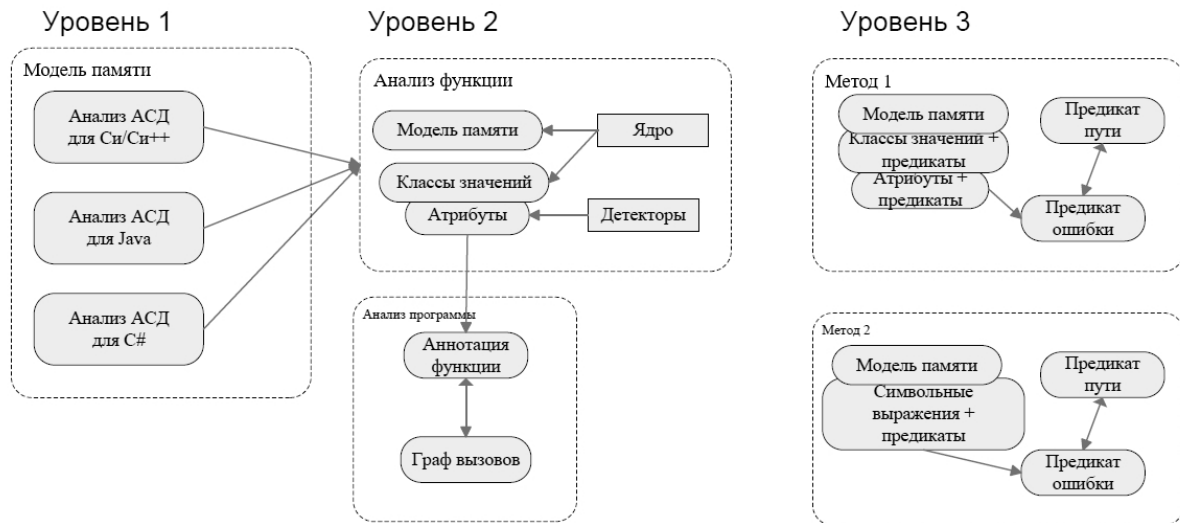


Рис. 1. Схема многоуровневого анализа исходного кода.

Предлагается выполнять многоуровневый статический анализ исходного кода, схема которого представлена на рис. 1. Первый уровень анализа выполняется для представления по типу абстрактного синтаксического дерева отдельно для каждого из поддерживаемых языков. Помимо детекторов, использующих те или иные обходы дерева, имеется возможность использования внутрипроцедурных анализов потока данных и управления. Для этого строится модель памяти, которая отражает операции с указателем для массивов и структур. Необходимый анализ возможных значений (в первую очередь целочисленных переменных и указателей) для внутрипроцедурных детекторов можно выполнять как в виде анализа потока данных (например, над интервалами значений), так и с использованием более сложных абстрактных доменов, в том числе предикатного домена.

Второй уровень анализа является межпроцедурным с контекстной чувствительностью и применением аннотаций функции (англоязычный термин — *summary*). Анализу предъявляется вся программа, в которой тела функций представлены в трехадресном представлении среднего уровня (по типу биткода LLVM или GCC GIMPLE), общем для всех поддерживаемых языков. Память моделируется так же, как и на первом уровне анализа, отслеживаются возможные значения переменных и множества ячеек памяти, на которые могут ссылаться указательные пе-

ременные. Дополнительно для отслеживаемых значений выполняется объединение одинаковых значений в классы эквивалентности (т.н. классы значений, или КЗ), аналогичные номерам переменных в алгоритме нумерации значений. В ходе вычислений при этом делается ряд предположений, ведущих к нестрогости анализа, т.е. потенциальному пропуску ошибок.

Детекторы конкретных дефектов вычисляют интересные их свойства классов значений или ячеек памяти (мы будем называть эти свойства атрибутами), задавая для них множества возможных значений, передаточные функции и функции объединения значений для точек слияния потока управления. Условием выдачи предупреждения об ошибке является истинность предиката ошибки, при построении которого используются вычисленные значения атрибутов.

По окончании внутрипроцедурного анализа создается аннотация функции, которая компактно описывает состояние анализа после выполнения инструкции возврата из функции. Межпроцедурный анализ состоит в однократном обходе графа вызовов программы от вызываемых функций к вызывающим. При обработке вызова функции ячейки памяти, соответствующие в её аннотации формальным параметрам, заменяются ячейками, моделирующими фактические параметры, и часть аннотации, которая построена для памяти вызываемой функции, достижимой из вызывающей через параметры и глобальные

переменные, переносятся в контекст вызывающей функции.

Третий уровень анализа обеспечивает чувствительность к путям выполнения в проводимом межпроцедурном анализе. Этот уровень можно организовать двумя способами. В первом способе в дополнение к отслеживанию классов значений внутрипроцедурный анализ использует символическое выполнение с объединением состояний. Классы значений являются в том числе символическими переменными, из предикатов с которыми анализ формирует булеву формулу, описывающую условие попадания в заданную точку функции (т.е. предикат пути). В модели памяти программы целочисленные значения отслеживаются, как и прежде, консервативно для определения необходимых ячеек памяти для массивов и указательной арифметики. Вдобавок к этому к значениям указателей и целых чисел приписываются предикаты, которые отражают условия, при которых будут приниматься эти значения, и выполняют изменения этих предикатов по мере прохождения по графу потока управления. Атрибуты, дополнительно заводимые детекторами ошибок, также получают предикаты. Выдача предупреждения об ошибке выполняется в результате проверки истинности предиката ошибки и его совместности с предикатом пути с помощью SMT-решателя. Для организации межпроцедурного анализа сформированные предикаты попадают в аннотацию функции, и их логические атомы, выражающиеся в терминах классов значений и ячеек памяти вызываемой функции, переписываются через контекст вызывающей функции при обработке инструкции вызова, совершенно аналогично основной части аннотации.

Другой способ организации чувствительного к путям анализа применим к языкам без указательной арифметики и других операций над указателями (C#, Java). Так как нет необходимости отслеживания целочисленных значений для отслеживания смещений при моделировании памяти, то в предлагаемой модели используемые смещения всегда известны. Значения ячеек памяти сразу являются символическими переменными, классы значений и их атрибуты не отслеживаются. Объединение состояний анализа, предикаты пути и ошибки строятся сразу над этими

переменными. По сути, часть функциональности анализа, выполнявшегося на втором уровне и выделявшего классы значений, переносится на SMT-решатель. Организация межпроцедурного анализа схожа с первым способом.

Далее в разделе 2 формулируется первый уровень анализа — уровень абстрактного синтаксического дерева, описывается модель памяти программы. В разделе 3 предлагается межпроцедурный анализ с контекстной чувствительностью на основе аннотаций, использующий в качестве внутрипроцедурного анализа комбинацию анализа потока данных и абстрактной интерпретации с выделением классов значений. В разделе 4 вводится чувствительный к путям анализ с использованием классов значений на основе формализма анализа второго уровня, а далее предлагается межпроцедурный анализ, непосредственно использующий символическое выполнение для достижения чувствительности к путям с привлечением модели памяти раздела 2, но без выделения классов значений. Раздел 5 завершает статью.

## 2. СТАТИЧЕСКИЙ АНАЛИЗ УРОВНЯ АБСТРАКТНОГО СИНТАКСИЧЕСКОГО ДЕРЕВА

Мы будем относить дефекты к проверяемым на этом уровне при следующих условиях: для обнаружения дефекта достаточно выполнить либо обход АСД функции, либо найти заданный шаблон на этом АСД; для обнаружения дефекта необходимо выполнить внутрипроцедурный анализ потока данных или управления, схожий по сложности с анализами, используемыми в компиляторах. Разберем отдельно организацию анализа на уровне АСД для обоих упомянутых случаев.

### 2.1. Организация обходов АСД

Для детекторов дефектов, выполняющих обход АСД, программа представляется в виде набора модулей трансляции, анализ каждого из которых выполняется отдельно. Каждый модуль представляется в виде пары  $M = \langle S, F \rangle$ , где  $S$  — таблица символов модуля, а  $F$  — множество функций (методов) модуля, каждая из которых представлена своим АСД. Таблица символов  $S$  является иерархической: корнем является

глобальная таблица символов (глобальные определения и объявления функций и переменных, пространств имен, классов — в зависимости от языка), а далее связи между таблицами “предок-потомок” соответствует связям между областями видимости в модуле трансляции.

Природа выполняемых детекторами проверок может быть совершенно различной, и, как представляется, для формализации стоит остановиться только на вопросе масштабируемости анализа. Для этого нужно потребовать, чтобы сложность алгоритмов детекторов была линейной от размера представления программы, т.е. от количества узлов АСД и элементов таблиц символов. Имея в виду это соображение, введем следующую классификацию детекторов:

1. **Детекторы 1 типа** выполняют обход только таблицы символов, но не АСД функций. В таблице символов выполняется поиск всех элементов заданного типа, после чего для каждого найденного элемента его “потомки” — элементы вложенных в него таблиц символов — могут посещаться не более одного раза. Как пример можно привести детекторы Си++-специфичных правил кодирования — о том, что классы должны содержать конструктор по умолчанию; в классе не должно быть публичных и приватных полей одновременно и т.п.
2. **Детекторы 2 типа** выполняют обход АСД функций в поисках узлов-операторов заданного типа, после чего для каждого найденного узла выполняется посещение не более чем константного количества других узлов АСД. Как представляется, данный тип детекторов наиболее распространен. Так, для безопасного кодирования часто используется правило о запрещении использования выделенного набора “опасных” функций, в языковых правилах — запрещение сравнивать переменные вещественного типа на точное равенство и т.п.
3. **Детекторы 3 типа** выполняют обход АСД функций в поисках узлов-операторов заданного типа, после чего для каждого найденного узла выполняется посещение всех его подузлов, но не более одного раза. Типичным примером такого детектора является

проверка ограничений, накладываемых на запись некоторого выражения — запрет использовать ассоциативность операций отношения (выражения вида  $a > b > c, a == b == c$ ), проверка потенциально ошибочных логических операций с одинаковыми аргументами (выражения вида  $a \parallel a$ ) и т.п.

4. **К детекторам 4 типа** относятся все детекторы, не подошедшие под условия одного из предыдущих типов. Как правило, такие детекторы после обнаружения в АСД узлов-операторов заданного типа просматривают подузлы найденного узла более одного раза, либо просматривают также и ряд узлов АСД, не являющиеся потомками найденного узла — родительские узлы, узлы-соседи того же уровня иерархии и т.д. В качестве примера можно привести проверку того, что индуктивные переменные цикла обновляются в теле цикла не более одного раза — количество посещений узлов АСД, относящихся к телу цикла, соответствует количеству индуктивных переменных.

**Утверждение 1.** Для данного модуля  $M = \langle S, F \rangle$ :

- детекторы 1 типа выполняются за время  $O(s)$ , где  $s$  — количество символов во всех таблицах символов модуля, тогда, когда при обработке элемента таблицы заданного типа в любой вложенной таблице символов дополнительно каждый символ просматривается детектором не более чем  $O(1)$  раз;
- детекторы 2 типа выполняются за время  $O(n)$ , где  $n$  — количество узлов в АСД данной функции;
- детекторы 3 типа выполняются за время  $O(n)$ , где  $n$  — количество узлов в АСД данной функции, тогда, когда при обработке узла заданного типа дополнительно в поддереве этого узла в АСД каждый узел просматривается детектором не более чем  $O(1)$  раз;
- детекторы 4 типа выполняются за время  $O(n)$ , где  $n$  — количество узлов в АСД данной функции, тогда, когда каждый поиск интересующего узла АСД занимает не более

чем  $O(1)$  шагов, при этом искомым детектором узлов не более  $O(f(n))$ , после чего количество дополнительно просматриваемых узлов АСД составляет не более чем  $O(n/f(n))$ , где  $f(n)$  — функция, характеризующая частоту встречаемости в функции искомым узлов.

**Доказательство** следует непосредственно из определения типов детекторов. Для детекторов первого типа сложность поиска заданного элемента по типу в данной таблице символов при использовании хеш-таблицы в качестве структуры данных составляет  $O(1)$ , а дальнейший просмотр вложенных таблиц для поиска элемента потребует  $O(t)$ , где  $t$  — количество таблиц, для первого элемента заданного типа из вложенной таблицы и  $O(1)$  для остальных элементов. Тем самым сложность поиска элемента заданного типа для всех элементов составит  $O(s+t)$ . После этого может быть дополнительно просмотрено еще  $O(s_i)$  элементов, где  $s_i$  — количество элементов в  $i$ -й таблице символов. Т.к. дополнительно каждый элемент просматривается не более одного раза, то для всех искомым элементов дополнительно будет просмотрено не более  $\sum_i s_i = s$  элементов, что составляет еще  $O(s)$  шагов. Итого имеем общую сложность в  $O(s+t)$  шагов. В наихудшем случае  $t = O(s)$ , тем самым сложность составляет  $O(s)$ .

Для детекторов второго типа обход АСД с поиском узлов нужного типа составит не более  $O(n)$  шагов, а т.к. на каждом успешном поиске дополнительно посещается не более  $O(1)$  узлов, то общая сложность составит  $O(n)$ , где  $n$  — количество узлов АСД.

Для детекторов третьего типа рассуждения схожи с таковыми для детекторов первого типа. Посещение всех нужных узлов займет не более чем  $O(n)$  времени, тогда как для каждого  $i$ -го узла дополнительный просмотр его подузлов займет не более  $O(n_i)$  шагов, а всего — не более чем  $\sum_i n_i = n$  шагов, т.е.  $O(n)$ .

Для детекторов четвертого типа, как оставшихся от классификации, сложно построить разумное ограничение на время выполнения. Можно лишь заметить, что если некоторая функция  $f(n)$  задает количество узлов АСД, которые должны быть протестированы детектором, то для достижения линейной сложности,

во-первых, на поиск узла нужно тратить в среднем не более  $O(1)$  шагов, а далее на каждый тест нужно расходовать в среднем не более чем  $O(n/f(n))$  шагов. Так как для проведения теста, как правило, нужно просмотреть некоторое количество других узлов, а сложность обработки одного узла константна, то по существу данное ограничение по времени работы ограничивает количество узлов, которые можно дополнительно просмотреть.

**Алгоритм 1.** Пусть  $f(n)$  значительно меньше  $n$ , тогда будем дополнительно хранить в каждом узле:

- индекс данного узла в порядке обхода в глубину, а также наименьший и наибольший индексы узлов этого поддерева;
- ссылку на следующий узел в АСД (в порядке обхода в глубину) того же самого типа;
- ссылки на следующие узлы в АСД (в порядке обхода в глубину) для остальных требуемых для детекторов типов узлов.

Тогда для обработки всех подузлов заданного типа:

- проверим хранимую ссылку на следующий узел искомого типа;
- пока индекс следующего узла лежит в пределах, заданных наименьшим и наибольшим индексами для узлов данного поддерева, будем обрабатывать необходимые узлы, переходя между ними по ссылкам 2) на следующий узел того же типа (аналогично, например, делается в прошитом двоичном дереве [1]).

В случае, когда количество искомым узлов  $f(n)$  сопоставимо с  $n$ , более подходящим вариантом поиска будет обычный обход АСД, не требующий дополнительных расходов памяти. При этом дополнительно для проведения теста детектор может просмотреть не более  $O(1)$  узлов.

Вообще говоря, для каждого подузла необходимо хранить ссылки на следующие в порядке обхода в глубину узлы всех требуемых типов, то есть дополнительно в каждом узле тратится линейный от количества поддерживаемых типов

узлов объем памяти. Можно улучшить ситуацию следующими способами:

1. Хранить не ссылки на узлы, а индексы следующих узлов нужного типа в порядке обхода в глубину. Для этого нужно дополнительно поддерживать отображение (хеш-таблицу), позволяющую получить по индексу ссылку на узел за  $O(1)$  времени. Для АСД функции в подавляющем большинстве случаев будет достаточно двух байт на индекс, тогда как ссылка в современных 64-битных системах займет 8 байт.
2. Воспользоваться тем, что для соседних узлов ссылки на следующие узлы нужного типа часто будут одинаковыми. Тем самым можно хранить эти ссылки только в некоторых узлах АСД (назовем такие узлы *базовыми*), а в остальных узлах хранить единственную ссылку на ближайший родительский базовый узел. Тогда возможно следующее улучшение алгоритма 1:

**Алгоритм 1'.** Для получения ссылки на следующий узел данного типа при хранении индексов в базовых узлах необходимо:

- получить (за  $O(1)$ ) ссылку на базовый узел
- получить ссылку на следующий узел данного типа для родительского узла;
- если индекс следующего узла находится в необходимых границах между наименьшим и наибольшим индексом для подузлов текущего узла, то можно итерироваться по узлам нужного типа по ссылкам 2), как в алгоритме 1;
- если индекс меньше наименьшего индекса, то нужно пропустить ряд узлов, проходя по ссылкам 2) до тех пор, пока индекс очередного узла не окажется в необходимых границах;
- если же индекс больше наибольшего индекса, то это означает, что подузлов искомого типа у текущего узла нет.

Базовыми узлами целесообразно сделать узлы, отвечающие за организацию потока управления (узлы ветвлений, циклов, операторов вы-

бора) и узлы составных операторов. Если провести аналогию с графом потока управления, то нужная информация будет храниться для каждого базового блока. Так как в типичных программах средний размер базового блока небольшой (5–10 инструкций), то количество дополнительных итераций по ссылкам 2) до попадания в интервал допустимых индексов узлов для этих случаев будет составлять  $O(1)$ .

## 2.2. Модель памяти программы и внутрипроцедурный анализ уровня АСД

Детекторы, для реализации которых требуется применение внутрипроцедурного анализа потока управления и данных, используют в качестве входных данных ту же пару  $M = \langle S, F \rangle$ , где  $S$  — таблица символов модуля, а  $F$  — множество функций модуля, но в этом случае представление функций в виде АСД дополняется графом потока управления:  $F = \{f : \langle A, G \rangle\}$ , где  $A$  — АСД функции, а  $G$  — ее граф потока управления. Базовые блоки графа потока сформированы из узлов АСД, соответствующих операторам языка программирования, не содержащим в себе переходов управления. При этом ссылки на родительские узлы АСД позволяют судить, например, о том, какие базовые блоки составляют данный узел-цикл или условие.

Для построения детекторов необходимо описать используемую ими модель памяти программы и её семантику, т.е. интерпретацию операторов языка в её терминах. Мы сконцентрируемся на описании подмножества операторов языка Си и модели памяти, поддерживающей все операции Си. При этом анализ программ на различных языках на уровне АСД может быть как полностью раздельным, так и опираться на некоторое “объединённое” АСД, узлами которого могут быть операторы всех поддерживаемых языков [2]. Этот вопрос ортогонален организации модели памяти для анализа.

Для моделирования памяти мы используем понятие ячеек памяти (также в [3]), которые принадлежат некоторым *областям памяти*. Концепция областей памяти следует идее классов памяти стандартов Си и Си++, и для целей анализа достаточно различать *статическую*, *автоматическую* (стек) и *динамическую* область памяти. Ячейки памяти принадлежат к одной из

описанных областей памяти и составляют четверку  $M = \langle R, S, O, P \rangle$ , где:

- $R$  является областью памяти;
- $P$  является *родительской* ячейкой памяти (например, ячейка для всего массива по отношению к элементу массива или ячейка полной структуры для поля структуры). Родительская ячейка также задает *базовый адрес* для данной ячейки памяти, т.е. либо виртуальный адрес начала стека для данной функции, глобальный указатель, либо начало динамически выделенного участка памяти. Поэтому отдельно хранить сущность, аналогичную базовому адресу, нет необходимости, достаточно ограничиться заданием родительской ячейки;
- $S$  и  $O$  задают размер ячейки памяти (в битах) и смещение относительно базового адреса.

При построении модели памяти предполагается, что в языковом АСД уже произошло распределение локальной памяти и вычисление размеров всех типов данных. Тем самым все смещения локальных переменных относительно начала фрейма функции, смещения полей структуры относительно ее базового адреса, размеры элементов массивов известны.

Опишем правила создания ячеек памяти для локальных, глобальных и динамических переменных. Ячейки памяти создаются “лениво” в момент первого доступа к ним. Необходимо рассмотреть следующие случаи:

1. Создается ячейка памяти, базовый адрес которой, размер и смещение относительно базового адреса для ее класса памяти известны статически (во время анализа);
2. Размер и/или смещение определяются динамически. Примерами таких ситуаций для языка Си с заранее неизвестным размером ячейки являются массивы переменной длины (VLA), ячейки, выделенные функцией `alloca`, внешние массивы, динамически выделенная память. Примеры заранее неизвестного смещения — это доступы к массивам с динамически вычисляемым индексом.

Для таких ситуаций сначала нужно предложить способ моделирования значений переменных программы (в основном, целочисленных), чтобы иметь возможность оценивать размер и смещение ячейки.

3. Базовый адрес ячейки памяти статически не определяется однозначно. Примерами таких ситуаций являются доступы к полям структуры по указателю, который может указывать на несколько потенциальных ячеек памяти для структур. В этом случае, помимо моделирования значений целочисленных переменных, необходимо также моделировать возможные значения указательных переменных, например, определяя для каждой множество ячеек памяти, на которые данная переменная может указывать (т.н. *points-to* множества).

Сначала мы рассмотрим случай 1) известного базового адреса, размера и смещения, потом предложим способы моделирования целочисленных значений и базовых адресов, а потом вернемся к рассмотрению создания новых ячеек памяти для случаев 2) и 3). Для простоты изложения, не теряя общности, будем рассматривать следующее множество выражений и операторов языка:

Итак, для полностью известных параметров ячеек памяти возможны следующие случаи:

1. Доступ к локальной переменной. Для каждой функции `foo` изначально создается ячейка памяти `MLFrameFoo`, представляющая фрейм функции в целом. В ячейке `MLFrameFoo`  $R = \text{Local}$ ,  $S$  — вычисленный компилятором минимальный размер фрейма функции (без учета массивов переменного размера),  $O = 0$ ,  $P = \text{MLFrameFoo}$ .

Далее, для ячейки памяти `MLLocal`, создаваемой для обрабатываемого доступа  $R = \text{Local}$ ,  $S$  — вычисленный компилятором размер типа переменной,  $O$  — вычисленное компилятором смещение данной локальной переменной относительно начала фрейма функции,  $P = \text{MLFrameFoo}$ .

2. Доступ к параметру функции. Создается ячейка памяти `MLParam` по аналогии с

$Expr := Operands \mid ArithExpr \mid AssignExpr \mid CompExpr \mid AddrExpr \mid FuncExpr$   
 $Operands := Constant \mid Variable$   
 $Variable := Scalar \mid Pointer \mid Array \mid Structure$   
 $ArithExpr := Expr + Expr \mid Expr - Expr$   
 $AssignExpr := Variable = Expr$   
 $CompExpr := Expr > Expr \mid Expr < Expr \mid Expr == Expr \mid Expr != Expr$   
 $AddrExpr := \&Expr \mid *Pointer \mid Var[Expr] \mid Var.Field \mid (type*)Expr \mid malloc(Expr)$   
 $FuncExpr := Variable(Expr_1, \dots, Expr_n) \mid Pointer(Expr_1, \dots, Expr_n) \mid return Expr$   
 $Stmt := Expr; \mid \{Stmt_1; \dots; Stmt_n; \} \mid if(Expr)Stmt; else Stmt; \mid while(Expr)Stmt;$

Рис. 2. Модельный язык.

ячейками памяти *MLLocal* для локальных переменных.

- Доступ к возвращаемому значению функции. Создается ячейка памяти *MLReturn* по аналогии с ячейками памяти *MLLocal*, отдельно для каждого вызова.
- Доступ к глобальной переменной. Поля ячейки памяти *MLGlob* заполняются следующим образом: R = Static, S — вычисленный компилятором размер типа переменной, O = 0, P — эта же ячейка памяти.
- Выделение динамической памяти (через вызовы функции *malloc* или подобных ей). Считается, что каждый новый вызов функции динамической памяти возвращает новую (не пересекающуюся с ранее выделенной) память. Поля ячейки памяти *MLHeap* заполняются следующим образом: R = Dynamic, S — размер участка памяти, соответствующий значению параметра функции *malloc*; O = 0, P — эта же ячейка памяти.
- Доступ к элементу составной структуры данных (полю структуры или элементу массива). Сначала создается ячейка памяти для всей структуры данных. Далее создается ячейка *MLElement* для элемента составного типа следующим образом: R — класс памяти родительской ячейки, S — вычисленный компилятором размер типа переменной; O — вычисленное компилятором смещение относительно родительской ячейки (смещение поля структуры либо смещение элемента массива относительно

начала массива), P — ячейка памяти для всей структуры.

Если для полей структур моделирование ячеек памяти выполняется точно, то для элементов массива задается константа *MaxSize*. В случае, когда индекс превышает значение *MaxSize*, новой ячейки памяти не создается, вместо этого используется специально заведенная ячейка *MLUndefElement*, в которой поле смещения O = -1. Это дает возможность ограничить точность моделирования для больших массивов.

Теперь для обработки случаев неизвестных статически базовых адресов и значений размеров и индексов введем отслеживание абстрактных значений для целочисленных и указательных переменных. Для целочисленных переменных будем пользоваться абстрактным доменом из интервалов значений [a, b], в которых границы интервалов — это целые числа или  $\pm\infty$ , и хранить для каждой ячейки памяти абстрактное значение *Val* из домена. Для указательных переменных будем хранить множества *PtTo* ячеек памяти, на которые могут указывать переменные.

Передаточные функции для интервального домена известны [4], и здесь мы приводить их не будем. Отметим только, что в случае перехода по условию неравенства интервалы не могут отразить этот факт, и требуется либо хранить объединение нескольких интервалов, либо отдельно отслеживать “выколотые точки” — значения, которым данная ячейка памяти не равна. Разыменование ячейки памяти обрабатывается как объединение интервалов всех возможных ячеек, на которые может указывать текущая. Для множеств *PtTo* инициализация заключается в присваивании специального значения *Undef*, обозначающего неизвестное множество. В адресной арифметике



$$\frac{a = p + \text{const}, PtTo(p) = \{ML_i\}}{PtTo(ML(a)) = \{ML_j : ML_j = Copy(ML_i) \wedge S_j = S_i + 8 \times \text{const} \times \text{sizeof}(*p)\}}.$$

Здесь  $Copy(ML_i)$  означает функцию, копирующую ячейку  $ML_i$ . Если вторым параметром сложения также является переменная или другое выражение, то оценивается его абстрактное значение  $Val$ , и результирующее смещение вычисляется в интервальной арифметике:

$$\frac{a = p + e, Val(e) = [a, b], Val(S_i) = [c, d], \text{sizeof}(*p) = sz}{Val(S_j) = [a + 8 \times sz * c, b + 8 \times sz * d]}.$$

При переходе после сравнения множества фильтруются:

$$\frac{if(p == q), PtTo(ML(p)) = \{ML_i\}, PtTo(ML(q)) = \{ML_j\}}{PtTo_{p==q}(ML(p)) = PtTo_{p==q}(ML(q)) = PtTo(p) \cap PtTo(q)},$$

$$\frac{if(p! = q), PtTo(ML(p)) = \{ML_i | i = \overline{1, m}\}, PtTo(ML(q)) = \{ML_j | j = \overline{1, n}\}}{m = 1 \rightarrow PtTo_{p!=q}(PtTo(q) \setminus PtTo(q)) \\ n = 1 \rightarrow PtTo_{p!=q}(ML(p)) = PtTo(p) \setminus PtTo(q)}$$

Взятие адреса и разыменование обрабатываются обычным образом, а слияние потока управления учитывает значение  $Undef$ :

$$\frac{n \geq 2, b_2, \dots, b_n \in Pred(b), PtTo_{b_i}(ML(a)) = \{ML_j^{b_i}\}}{PtTo_b(ML(a)) = Unfty_i(PtTo_{b_i}(ML(a)))},$$

где

$$Unify(PtTo_a, PtTo_b) = \begin{cases} \{Undef\}, & Undef \in PtTo_a \vee Undef \in PtTo_b \\ PtTo_a \cup PtTo_b, & \text{иначе} \end{cases}.$$

Если размер ячейки или ее смещение относительно родительской ячейки не являются константами, то вычисляется абстрактное значение смещения  $Val_o$  и/или размера  $Val_s$  создаваемой ячейки для данной точки программы; просматриваются все ранее созданные ячейки памяти для данной родительской ячейки (начала фрейма функции, массива или заданного вызова функции выделения динамической памяти). Если находится такая ячейка памяти  $ML$ , что для нее  $Intersect(S_{ML} + O_{ML}, Val_s + Val_o) \neq \emptyset$ , то новая ячейка памяти не создается, а вместо этого обновляются значения смещения и размера старой ячейки:  $S_{ML} = Unify(S_{ML}, Val_s)$ ,  $O_{ML} = Unify(O_{ML}, Val_o)$ .

Для всей функции ячейки памяти, абстрактные значения  $Val$  и множества  $PtTo$  вычисляются с помощью следующего алгоритма.

**Алгоритм 2.** Входные данные: функция  $f = \langle A, G \rangle$ . Выходные данные:  $V = \{Val_i\}$ ,  $P = \{PtTo_i\}$  — вычисленные абстрактные значе-

ния  $Val$  и множества  $PtTo$  для всех ячеек памяти функции в каждой точке (операторе) функции;  $M = \{Expr \rightarrow ML\}$  — соответствие между выражениями, именующими память, и ячейками памяти.

1. Инициализация: положить  $M_{Entry} = \{ML_{static}\}$ , где  $ML_{static}$  — ячейки памяти для глобальных и статических переменных.
2. Структурный анализ: построить дерево управления, состоящее из выделенных в графе потока управления шаблонов вида *Block*, *IfThen*, *IfThenElse*, *While*. Шаблоны определяются так:
  - *Block*: последовательность регионов (участков графа потока управления) с одним входом и одним выходом, в которой каждый регион является единственным последователем предыдущего региона и единствен-

ным предшественником следующего региона;

- *IfThen, IfThenElse, While*: регионы с одним входом и одним выходом, соответствующие одноименным узлам абстрактного синтаксического дерева.

Выделение шаблонов проводится напрямую по узлам-операторам АСД либо классическим алгоритмом структурного анализа [sharir].

3. Основной цикл: обойти дерево управления в глубину, узлы на одном уровне обходятся в порядке следования в функции. Обработка конкретных шаблонов выполняется так:

- *Block*: регионы обрабатываются последовательно. Если регион является базовым блоком, то для каждого оператора *Stmt* базового блока последовательно применяются описанные передаточные функции  $F_{Stmt}$ , которые обновляют множество  $M$ , а далее строят множества  $V_{Out}$ ,  $P_{Out}$  в точке после оператора по множествам  $V_{In}$ ,  $P_{In}$  в точке до оператора.
- *IfThen, IfThenElse*: обрабатываются регионы, соответствующие условию в операторе, регион *Then* и при наличии регион *Else*, далее проводится слияние вычисленных данных с помощью описанных функций *Unify*. А именно, для всех ячеек памяти  $ML$ ,  $Val_{IfThenOut}(ML) = Unify(Val_{Expr=false}(ML), Val_{ThenOut}(ML), PtTo_{IfThenOut}(ML)) = Unify(PtTo_{Expr=false}(ML), PtTo_{ThenOut}(ML))$ , здесь  $Val_{Expr=false}$  соответствует абстрактному значению в случае невыполнения условия в условном операторе. Аналогично в случае шаблона *IfThenElse* объединяются выходные значения для регионов *Then* и *Else*.
- *While*: обрабатывается условие цикла предложенными передаточными функциями. Выбирается число  $k \geq 3$  (количество анализируемых итераций цик-

ла), полагается  $i = 1$  (номер текущей итерации). Далее:

- о если  $i = 1$ , то полагается  $Val_{BodyIn} = Val_{Expr=true}$ ,  $PtTo_{BodyIn} = PtTo_{Expr=true}$  и выполняется анализ региона, соответствующего телу цикла,  $i$  увеличивается на единицу;
- о если  $i > 1$ , то перед началом анализа тела цикла выполняется слияние абстрактных значений и множеств  $PtTo$  вдоль обратной дуги цикла: для всех ячеек памяти  $ML$   $Val_{BodyIn}(ML) = Unify(Val_{BodyOut}(ML), Val_{WhileIn}(ML), PtTo_{BodyIn}(ML)) = Unify(PtTo_{BodyOut}(ML), PtTo_{WhileIn}(ML))$ ; после этого накладываются ограничения из обработки условий тела цикла (т.е. считаются  $Val_{Expr=true}^i$  и  $PtTo_{Expr=true}^i$  для очередной итерации), проходит анализ тела цикла, и  $i$  увеличивается на единицу;
- о при  $i = k - 1$  перед началом анализа тела цикла выполняется слияние значений и последующее их ограничение из обработки условия, аналогично предыдущему случаю. Далее выполняется анализ тела цикла, в котором при вычислении абстрактного значения  $Val_i$  для ячейки памяти  $ML$  сначала вычисление выполняется обычным образом, а затем применяется расширяющий оператор  $\nabla$  [6]:  $Val_i(ML) = \nabla(Val_{i-1}, Val_i)$ , где  $Val_{i-1}(ML)=[a,b], Val_i(ML)=[c,d]$   
 $\nabla(Val_{i-1}, Val_i)=[c < a? -\infty : a, d > b? +\infty : b]$ . Аналогично при вычислениях множеств  $PtTo$  выполняется расширение для них, в котором при несовпадении множеств  $PtTo_{i-1}$  и  $PtTo_i$  устанавливается  $PtTo_i = Undef$ . Наконец,  $i$  увеличивается на единицу;
- о при  $i = k$  перед началом анализа тела цикла выполняется слияние

значений и последующее их ограничение из обработки условия, аналогично предыдущему случаю. Далее происходит последняя итерация анализа, после которой выполняется окончательное слияние результатов анализа тела цикла и значений, приходящих в точку слияния по дуге, минуящий цикл:  $Val_{WhileOut}(ML) = Unify(Val_{bodyOut}(ML), Val_{WhileIn}(ML))$ ,  $PtTo_{WhileOut}(ML) = Unify(PtTo_{bodyOut}(ML), PtTo_{WhileIn}(ML))$  для всех ячеек памяти  $ML$ . Наконец, вычисляются окончательные значения  $Val_{Expr=true}^k$  и  $PtTo_{Expr=true}^k$  как результат анализа цикла.

**Утверждение 2.** В предположении отсутствия псевдонимов в параметрах анализируемой функции  $f$  алгоритм 2 корректно вычисляет абстрактные значения и множества  $PtTo$  ячеек памяти, т.е. множества конкретных значений и конкретных целей указателей являются подмножествами соответствующих вычисленных абстрактных множеств.

Для доказательства достаточно заметить, что в предложенной модели инициализация прежде не встречавшихся ячеек памяти выполняется в виде создания новой ячейки памяти. Т.к. параметры функции при распределении локальной памяти получают непересекающиеся участки, то данное предположение означает, что между ними не будет псевдонимов. В случае, если параметры имеют указательный тип, их разыменование аналогичным образом приведет к созданию новых непересекающихся ячеек памяти. Абстрактные значения для новых ячеек памяти являются максимально широкими (верхними элементами решеток), тем самым они консервативно корректны.

Далее, при анализе операторов из базового блока последовательно применяются передаточные функции, каждая из которых сохраняет корректность на выходе из оператора при условии корректности значения на входе: это непосредственно следует из свойств абстрактного домена интервалов значений [кз], а для множеств  $PtTo$

в этом легко убедиться непосредственно. Для условных операторов дополнительно используется лишь слияние абстрактных значений вдоль нескольких путей выполнения, которое также сохраняет корректность, поскольку результирующее абстрактное значение консервативно отражает все возможные варианты значений, полученные вдоль отдельных путей. Наконец, при обработке циклов для получения консервативно корректного значения на предпоследней итерации анализа используется оператор расширения, а последняя итерация анализа тела цикла ограничивает абстрактные значения на входе в цикл, играя роль сужающего оператора, корректность которого для домена интервалов также известна [4].

**Утверждение 3.** Алгоритм анализа 2 завершается и выполняется за время  $O(nk^d)$ , где  $n$  — количество инструкций в функции,  $k$  — количество итераций анализа цикла,  $d$  — максимальная глубина цикла.

**Доказательство** непосредственно следует из построения алгоритма анализа — инструкции внешнего цикла из гнезда циклов будут анализироваться  $k$  раз, следующего по вложенности —  $k^2$  раз, и так далее, а количество инструкций внутреннего цикла заведомо не превосходит  $n$ . ■

В распространенных программах глубина циклов обычно невелика. Так, по нашим данным, среднее значение метрики вложенности управляющих операторов (включающей в себя и условные операторы) для исходного кода ОС Tizen 3.0 составляет 2.16, что дает возможность предполагать, что обычно анализ будет выполняться за линейное время относительно количества инструкций функции ( $3^{2.16} \approx 10.73$ ). Максимальная глубина циклов может значительно превосходить это значение — так, в [5] приведена статистика тестов SPEC CPU INT 2000, из которой следует, что в 5 из 11 тестов максимальная глубина циклов равна 10.

### 3. МЕЖПРОЦЕДУРНЫЙ КОНТЕКСТНО-ЧУВСТВИТЕЛЬНЫЙ АНАЛИЗ

Второй уровень анализа требуется для поиска критических дефектов — таких, как переполнение буфера, разыменования нулевого указателя, ошибки управления динамической памятью,

утечки ресурсов — с хорошим качеством. Это означает межпроцедурный анализ, обладающий хотя бы некоторой контекстной чувствительностью (т.е. учитывающий влияние контекста вызова функции на ее выполнение) и чувствительностью к путям выполнения (т.е. отслеживающий, при каких именно условиях, направляющих выполнение по разным путям, будут верны те или иные утверждения о потоке данных программы). Тем не менее, как было сказано в начале главы, межпроцедурный анализ без чувствительности к путям является полезным для некоторых детекторов. Поэтому в данном разделе сначала мы опишем внутрипроцедурный анализ на основе классов значений и организацию межпроцедурного анализа, а варианты привнесения в анализ чувствительности к путям обсудим в разделе 4.

Модельным языком для наших построений останется тот же язык на рисунке 2. Однако внутреннее представление является более низкоуровневым (трехадресным) — в нем сложные выражения языка понижены до четверок  $\langle Op, VarDest, VarLeft, VarRight \rangle$ . Для этого вводятся *псевдореестры* — неадресуемые временные типизированные переменные с единственным определением (обозначаем их как  $@Variable_{index}$ ). Будем считать, что на этом этапе абстрактное синтаксическое дерево уже недоступно, однако по необходимости сохранена высокоуровневая информация (о типах, соответствия инструкций строкам исходного кода, виртуальности вызовов и т.п.). Анализ получает на вход множество функций  $F = \{f : \langle S, G \rangle\}$ , где  $S$  — определение функции (имя, тип возвращаемого значения, имена и типы параметров),  $G$  — граф потока управления функции из описанных инструкций внутреннего представления, и множество глобальных переменных  $Glob = \{g : \langle S, I \rangle\}$ , где аналогично  $S$  — определение переменной, т.е. ее имя и тип,  $I$  — начальное значение переменной. Заметим, что у различных языков могут быть свои аналоги глобальных переменных, однако общая схема рассуждений при этом сохранится.

Опишем сначала алгоритм анализа самого верхнего уровня, а потом обсудим его части в дальнейших подразделах. Целью данного алгоритма является выполнить анализ всех функций

алгоритмом 2 с учетом эффектов от вызовов функций с контекстной чувствительностью, то есть построить модель памяти программы и вычислить основные факты о ее переменных (абстрактные значения, потенциальные цели указателей) для всех точек программы. Этот алгоритм составляет т.н. *ядро* анализа. Вопросы написания *детекторов* конкретных ошибочных ситуаций с помощью данных, собранных ядром, и дополнительно вычисленных атрибутов рассмотрены в разделе 5.

**Алгоритм 3.** Межпроцедурный анализ на основе аннотаций.

1. Построение графа вызовов программы. В графе вызовов вершинами являются анализируемые функции, а дуги показывают связи по вызовам между ними. Бывают дуги двух типов: прямые, соответствующие вызовам по имени, и *косвенные*, соответствующие вызовам по указателю (в том числе для виртуальных функций). Требуется выполнить анализ указателей на функцию для построения достаточно точного графа вызовов.
2. Разрыв циклов. В графе вызовов определяются сильно связанные компоненты, после чего из каждой компоненты удаляется произвольная дуга (если возможно, косвенная). Это нужно для получения ациклического графа, который можно проанализировать за один проход.
3. Межпроцедурный анализ. Обойти граф вызовов в обратном топологическом порядке (от листьев к верхним уровням). Для каждой обходимой вершины:
  - 3.1. Если вершина соответствует известной функции (для которой имеется внутреннее представление), то проводится внутрипроцедурный анализ этой функции по алгоритмам, описанным далее в разделе 3.1. По окончании анализа создается аннотация функции (результаты ее анализа) согласно подходу, описанному в разделе 3.2.
  - 3.2. Если вершина соответствует функции, для которой нет тела, но она является известной анализу (например,

функцией стандартной библиотеки), то ее аннотация также предполагается известной. Запись аннотации стандартных функций мы будем называть *спецификацией функции*. Возможные способы описания спецификаций выходят за рамки данной статьи.

Для прямых вызовов (не по указателю) построение графа вызовов не представляет сложностей. Достаточно создать вершину в графе для каждой функции и обойти все тела функций для поиска инструкций вызова. Если вызываемая функция также находится в графе вызовов, то между соответствующими вершинами проводится прямая дуга. Если же функции нет, то создается специальная вершина для функции без тела и помечается как неизвестная функция, а потом проводится дуга.

Вызовы по указателю можно трактовать как вызовы неизвестных функций. Однако точность анализа от этого страдает. Целесообразно провести возможно быстрый анализ для уточнения потенциальных целей косвенных вызовов. Для этого предлагается обойти тела функций, обращая внимание только на инструкции, оперирующие с указателями на функцию — взятие адреса функций, присваивание указателей, вызовы по указателю, присваивание переменных составных типов (массивов, структур), которые содержат указатели на функцию — и вычислить множества *PtTo* только для таких указателей. Далее для вызовов функций по указателю проводятся косвенные дуги до всех функций, попавших в соответствующее множество *PtTo*.

### 3.1. Внутрипроцедурный анализ

Целью внутрипроцедурного анализа является построение для заданной функции программы модели памяти и вычисления основных фактов о ее потоке данных. Моделью памяти для нас будут являться уже описанные в разделе 2 ячейки памяти. Правила создания ячеек памяти необходимо дополнить для введенных нами псевдорегистров — будем считать, что создается типизированная ячейка памяти со специальным классом памяти ( $R = Pseudo$ ) и инициализированным размером ( $S = 8 \times \text{sizeof}(type)$ ), нулевым смещением и отсутствием родительской ячейки.

Мы будем отслеживать значения с помощью концепции *классов значений* (КЗ). Классы зна-

чений предназначены для отслеживания эквивалентности значений переменных, что помогает повысить точность анализа без привлечения SMT-решателей [6, 7]. Поэтому с каждой ячейкой памяти в каждой точке программы ассоциируется класс значений:  $VCs: ML \times I \rightarrow VC$ , который выполняет роль абстрактного значения. Правила присвоения нового класса значений схожи с правилами заведения номера значений в нумерации значений, основанной на хешировании — одинаковый класс значений для двух ячеек памяти означает, что анализ смог доказать, что в них хранится одно и то же значение. Оценки этого значения для целочисленных и указательных переменных становятся свойствами классов значений, а не самих ячеек памяти.

Классы значений распространяются через инструкции следующим образом: присваивание константы — заводится класс значений, соответствующий константе, и запоминается в хеш-таблице классов значений; присваивание переменной — класс значений ячейки памяти для целевой переменной выставляется по классу ячейки для правой части, в хеш-таблицу заносится новый класс значений для целевой ячейки; арифметические операции — 
$$\frac{a=b \text{ op } c, op \in \{+, -\}}{VC(ML(a)) = VLookup(op, VC(ML(b)), VC(ML(c)))}$$
. Здесь функция *VLookup* проверяет наличие записи в хеш-таблице о том, что данная операция с данными классами значений уже была вычислена и ей был присвоен класс значений. В случае положительного ответа возвращается тот же класс значений, иначе заводится новый класс (отличный от всех имеющихся) и запоминается в хеш-таблице. При слиянии потока управления, если вдоль различных путей выполнения, прошедших в данную точку, ячейка памяти имела одинаковый класс значения, то он же и остается после слияния, иначе заводится новый класс. Разыменование выполняется аналогично.

Основными атрибутами, вычисляемыми ядром, являются уже упомянутые интервалы значений *Val* и множества потенциальных целей указателей *PtTo*. Передаточные функции для них остаются прежними с той лишь разницей, что теперь для их получения по ячейке памяти необходимо сначала узнать класс значений, хранящийся в ячейке, а потом по этому классу получить искомый атрибут. Формулы для пере-

дательных функций получают соответствующие механические изменения. Нужно отметить, что в случае слияния классов значений функцией *Unify*, если все классы значений были одинаковыми, то и атрибуты у них одинаковые — оставляется как старый класс значения, так и прежние атрибуты. Иначе заводится новый класс значений, а его атрибуты получаются слиянием атрибутов соответствующих классов. Так, при обработке разыменований указателей в правой и левой части присваивания (загрузка и запись по указателю соответственно) в случае, когда множество *PtTo* содержит единственный элемент, то атрибуты только его класса значений участвуют в вычислениях. То есть если проводится запись, то обновляется только конкретная ячейка памяти и ее класс значений (т.н. *сильное* обновление). Если же элементов несколько, то при обработке инструкции вида  $*p = expr$  выполняется *слабое* обновление — для каждого класса значений, который связан с ячейками памяти — потенциальными целями указателя  $p$  — выполняется объединение функцией *Unify* со значением, полученным из *expr*.

**Алгоритм 4.** Внутрипроцедурный анализ функции с применением классов значений. Входом алгоритма является функция  $f : \langle S, G \rangle$ . Выходом являются посчитанные функции модели памяти  $M : LValue \rightarrow ML$ , классы значений и их атрибуты  $VCs : ML \times I \rightarrow VC$ ,  $Attrs : VC \times AttrName \rightarrow Attr$  ( $AttrName \in \{Val, PtTo\}$ , другие атрибуты могут быть добавлены детекторами).

1. Инициализация глобальных ячеек памяти предполагается выполненной межпроцедурным алгоритмом (соответствующий шаг 1 алгоритма 2 пропускается).
2. Выделение регионов выполняется обычным структурным анализом.
3. Обход функции по дереву управления выполняется аналогично шагу 3 алгоритма 2 со следующими отличиями:
  - 3.1. Новые классы значений создаются для новых ячеек памяти.
  - 3.2. При обработке инструкций сначала обновляются классы значений для ячеек

памяти, а потом обновляются их атрибуты через передаточные функции.

- 3.3. При обработке ветвлений и циклов перед слиянием атрибутов (интервалов значений и множеств *PtTo*) сначала сливаются классы значений, а потом их атрибуты, как было описано в алгоритме 2.
- 3.4. Расширяющий оператор  $\nabla$  на предпоследней итерации цикла по умолчанию не применяется (чтобы получить более точные результаты анализа первых  $k$  итераций цикла, при этом, возможно, не учтя эффекты поздних итераций).

**Утверждение 4.** В предположении отсутствия псевдонимов в параметрах анализируемой функции  $f$  алгоритм 4 корректно вычисляет классы значений, абстрактные значения и множества *PtTo* ячеек памяти (т.е. множества конкретных значений и конкретных целей указателей являются подмножествами соответствующих вычисленных абстрактных множеств, а классы значений совпадают только тогда, когда совпадают конкретные значения соответствующих ячеек памяти) для путей выполнения через функцию, проходящих по циклам не более  $k$  раз, где  $k$  — количество анализируемых итераций.

**Доказательство** аналогично доказательству утверждения 2 с той разницей, что при отсутствии расширяющего оператора анализ не пройдет по путям, которые итерируются по циклам более чем  $k$  раз, тем самым вычисленные абстрактные значения могут не отражать конкретных значений, принимаемых переменными на этих путях. Совпадение классов значений происходит только тогда, когда данные значения были получены с помощью одной и той же цепочки вычислений (что гарантируется процедурой выбора классов значений при обработке инструкций), при этом в случае слияния потока управления эти значения были одинаковы по всем путям выполнения (иначе был бы сгенерирован новый класс значения).■

### 3.2. Вычисление и использование аннотаций функции

Общепринятым способом организации масштабируемого межпроцедурного анализа явля-

ется использование аннотаций функций. Смысл аннотации в компактной записи, во-первых, влияния вызываемой функции на внешнюю память, и, во-вторых, влияния контекста вызова функции на ее выполнение. При обработке вызова функции используется созданная аннотация, повторного анализа функции не происходит, что позволяет достичь масштабируемости.

Для построения аннотаций необходимо разрешить следующие вопросы:

- что попадает в аннотацию функции в результате анализа,
- каким образом аннотация используется при анализе вызова функции,
- степень контекстной чувствительности, достигаемой аннотациями (т.е. сколько контекстов вызова описывает конкретная аннотация). Этот вопрос по сути является ключевым — ответ на него задает конкретные алгоритмические решения для остальных вопросов.

Выбирая степень контекстной чувствительности анализа, заметим, что создание отдельной аннотации для каждого контекста приводит к экспоненциальному росту числа аннотаций. Использование отдельной аннотации для некоторой группы контекстов (например, популярное решение создавать аннотацию для каждой группы путей в графе вызовов, у которых последние  $k \geq 2$  вызовов совпадают) — это способ формально описать интуитивное представление о поведении программы: более вероятно, что вызываемая функция будет вести себя по-разному для “значительно” различающихся контекстов. Но зададимся вопросом — что даёт для проводимого нами анализа создание отдельной аннотации для некоторого контекста? Очевидно, что хочется добиться более точного вычисления абстрактных значений и других атрибутов ячеек памяти, создаваемых и модифицируемых функцией. Интуитивно это может происходить в том случае, когда на вход функции подаются различные значения, учет которых при вычислениях приводит к различным ответам для оцениваемых атрибутов. Если удастся *параметризовать* влияние контекста вызова в аннотации, то есть выразить зависимость

вычисленных значений от входных значений функции, то такая параметризованная аннотация может одна успешно заменить множество вычисленных обычным способом аннотаций для различных контекстов вызова.

Формализуем это соображение, введя понятие внешних ячеек памяти и предложив учитывающий их алгоритм внутривычислительного анализа. *Внешними* ячейками памяти *EML* для данной функции будут ячейки памяти, соответствующие формальным параметрам функции и глобальным переменным (и другой статической памяти). Аналогично, *внешними классами значений* назовем те классы значений, которые хранятся во внешних ячейках памяти перед началом анализа функции:  $EVC \in Vcs$ ,  $EVC = VC(EML, f_{Entry})$ . *Внешними абстрактными значениями* (интервалами значений, множествами *PtTo* либо атрибутами детекторов) назовем абстрактные значения внешних классов значений. Фактическим значением в данном случае будем называть абстрактное значение, приближающее значение параметра функции или глобальной переменной в данном контексте вызова.

Модифицируем передаточные функции для абстрактных значений следующим образом. Каждый раз, когда в вычислениях передаточной функции участвует внешнее абстрактное значение, и соответствующая функция является ассоциативной, вместо применения конкретного значения по умолчанию символично подставляется внешнее абстрактное значение. Если же передаточная функция не ассоциативна, то подставляется конкретное значение по умолчанию. Например, если вычисляется передаточная функция для выражения  $x+1$ , где  $x$  — параметр функции, то вместо вычисления интервала  $[-\infty, +\infty] + [1, 1] = [-\infty, +\infty]$  записывается интервал  $Val(EVC(EML(x))) + [1, 1]$ . Дальнейшее применение передаточных функций к таким значениям выполняет вычисления над конкретными интервалами, если это возможно, а символичные вхождения внешних значений оставляет нетронутыми (например, последующее вычисление  $x+2$  приводит к интервалу  $Val(EVC(EML(x))) + [3, 3]$ ).

Идея данного подхода заключается в том, чтобы сохранить все операции, проведенные переда-

точными функциями над внешними абстрактными значениями, подставляя их символично, вместо того, чтобы потерять точность представления значений, оперируя неизвестными значениями по умолчанию. Тогда при сохранении аннотации функции свойства соответствующих классов значений будут вычислены не до конца, а выражены символично через внешние абстрактные значения. Далее при обработке аннотации в конкретном контексте вызова символическое внешнее абстрактное значение заменяется на фактическое, и производится довычисление значений, чтобы как можно более точно охарактеризовать классы значений и ячейки памяти в точке вызова.

Из приведенных рассуждений непосредственно следует

**Утверждение 5.** Если алгоритм 4 используется с передаточными функциями, в которых внешние абстрактные значения применяются символично, и все передаточные функции с внешними значениями и функции объединения значений являются ассоциативными, то при применении аннотации функции, подставляя фактическое значение на место внешнего, после вычислений будет получено то же значение, которое было бы вычислено при изначальном применении алгоритма 4 при задании этого фактического значения как исходного.

Для доказательства достаточно заметить, что окончательные значения, сохраненные в аннотации, будут выражены через некоторую комбинацию  $f_1, f_2, \dots, f_n$  передаточных функций и функций объединения, в которых некоторые параметры будут внешними значениями, а некоторые — конкретными значениями, в свою очередь вычисленные над конкретной частью сохраненных значений. Т.к. все примененные функции являются ассоциативными, то изменение порядка их вычисления (сначала только над конкретной частью значений, потом над фактическими значениями, подставленными вместо внешних) не влияет на окончательный ответ по сравнению со способом его вычисления в естественном порядке анализа (как если бы все значения были известны заранее). ■

Заметим, что функции объединения для интервалов значений и множеств  $PtTo$  являются ассоциативными, тем самым введение внешних

классов значений помогает восстановить потери точности, происходящие при обработке точек слияния потока управления.

Аннотация функции по сути является частью отображения  $M$  и  $VCs$  в точке возврата из функции. Однако сохранение всего отображения будет содержать избыточное количество информации: данные, характеризующие локальную для функции память, будут неактуальны в контексте вызова, т.к. вызывающая функция никак не может их использовать. Требуется сохранить только классы значений и их атрибуты для ячеек памяти, которые могут быть доступны через внешние ячейки — те ячейки, которые либо являются внешними, либо дочерними ячейками по отношению к внешним, либо входят в множества  $PtTo$  для внешних ячеек через один либо несколько уровней косвенности. Как правило, количество сохраняемых уровней косвенности ограничивают, чтобы избежать чрезмерного роста аннотации. Эти соображения дают нам следующую

**Алгоритм 5.** Построение аннотации функции.  
Вход: вычисленные отображения  $M$  и  $VCs$ , константа  $md \geq 3$ . Выход: построенное резюме  $Ann \in VCs$ .

1. Добавить в  $Ann$  внешние ячейки памяти, ячейки памяти для возвращаемых значений и их классы значений в точке возврата из функции, положить  $iter = 0$ .
2. Для всех ячеек памяти  $ML$ , добавленных в  $Ann$  и не обработанных ранее:
  - Пометить ячейку  $ML$  как обработанную.
  - Добавить в аннотацию все ячейки памяти, для которых  $ML$  является родительской ячейкой, и их классы значений в точке возврата из функции, пометить все эти ячейки как обработанные.
  - Если  $ML$  соответствует указательному типу и имеет непустое множество  $PtTo$ , то добавить в аннотацию все ячейки из множества  $PtTo(ML)$  и их классы значений в точке возврата из функции, пометить эти ячейки как необработанные.
3. Положить  $iter = iter + 1$ . Если  $iter = md$ , то остановиться, иначе перейти к шагу 2.



Для применения аннотации необходимо построить отображение между ячейками памяти и классами значений, с одной стороны, из контекста вызывающей функции, с другой стороны, сохраненными в резюме. Процесс построения аналогичен алгоритму 5 и заключается в параллельном разворачивании ячеек памяти по уровням косвенности в сохраненной аннотации и в контексте вызова и сопоставлении этих ячеек друг другу. На первом шаге алгоритма внешние ячейки памяти из аннотации сопоставляются с ячейками памяти фактических параметров, глобальных переменных и возвращаемых значений в контексте вызова. На следующих итерациях по уровням косвенности сопоставляются дочерние ячейки памяти, для ячеек памяти из множества *PtTo* в вызывающую функцию переносятся новые ячейки, созданные в вызываемой. Если ячейка памяти, для которой ищется сопоставляемая ячейка в аннотации, уже была поставлена в соответствие некоторой ячейке в вызывающей функции, то их классы значений и их атрибуты объединяются функциями *Unify* (в этом случае оказывается нарушенным предположение об отсутствии алиасов между параметрами функции, которое делалось в ходе выполнения межпроцедурного анализа алгоритмом 4). После окончания процесса сопоставления происходит пере-вычисление атрибутов классов значений (интервалов значений и множеств *PtTo*), которые зависели от внешних значений, подставлением их фактических значений и повторным применением передаточных функций и функций объединения.

#### 4. МЕЖПРОЦЕДУРНЫЙ АНАЛИЗ С ЧУВСТВИТЕЛЬНОСТЬЮ К ПУТЯМ

На третьем уровне анализа выполняется поиск критических ошибок общего характера (переполнения буфера, утечки ресурсов, разыменования нулевого указателя и т.п.), для которых можно назвать справедливыми следующие два утверждения. Во-первых, такие ошибки сложно искать с помощью “узкоспециализированных” шаблонов в коде, которыми пользуется первый уровень анализа. Можно найти ряд вариантов появления таких ошибок, но общие ситуации будут все время ускользать от анализатора (конечно, при условии сохранения высокого уровня истинных срабатываний). Например, анализ на

уровне АСД может искать специфические переполнения буфера на один элемент при использовании функции *strncat* и известных во время компиляции размеров и длин строк, но не сможет найти переполнение, возникшее при неверном доступе к обычному массиву из-за неверного вычисления неконстантного индекса. Во-вторых, эти ошибки случаются лишь на небольшой доле путей выполнения, тем самым для достижения хорошего качества анализа нельзя консервативно выдавать ошибки при возможно несовместных условиях переходов. Тем самым поддержка чувствительности к путям в анализе становится неизбежной.

##### 4.1. Чувствительный к путям анализ с классами значений

Для адаптации анализа раздела 3 для чувствительности к путям необходимо рассмотреть следующие изменения:

- Построение предиката пути. Для каждой точки программы внутрипроцедурный анализ будет строить булеву формулу, которая является истинной при условии достижения потоком выполнения этой точки. Предикат пути используется в дальнейшем как часть формулы, совместность которой необходимо проверить для выдачи сообщения об ошибке, и для отслеживания предикатов атрибутов, в том числе основных — множеств *PtTo* и интервалов значений.
- Доработка модели памяти. Для ячеек памяти, составляющих множества *PtTo*, дополнительно будут отслеживаться предикаты, при условии истинности которых указательная переменная будет указывать на данную ячейку памяти. Для интервалов значений сохранится консервативное отслеживание через функцию *Unify* для вычисления неконстантных смещений в ячейках памяти.
- Отслеживание предикатов для классов значений. При объединении классов значений кроме выделения нового класса записываются предикаты, при которых данный класс значений совпадает с некоторым другим. Эти предикаты можно использовать в дальнейшем для уточнения возможного значения, приписанного данному классу значений.

- Отслеживание предикатов для атрибутов детекторов. Если детектор отслеживает некоторый атрибут класса значения, то при прохождении анализом некоторой инструкции детектор может, аналогично предикату пути, конструировать формулу, истинность которой означает, что атрибут принимает заданное значение. Атомами этой формулы являются классы значений, выступающие как символьные переменные.

Основная логика чувствительного к путям анализа заключается в том, каким именно образом конкретный детектор строит формулу ошибки (предикат пути строится анализом централизованно для всех детекторов). В отличие от анализа раздела 3, здесь алгоритм детектора играет большую роль, чем алгоритмы ядра.

Предикат пути *Curr* отслеживается дописыванием к нему элементарных формул сравнений классов значений и объединения отдельных предикатов путей в точке слияния потока управления следующим образом:

- При входе в функцию  $Curr = true$ .
- При переходе через условный оператор вида  $if(a == b)$  в базовый блок, соответствующий истинному значению условия, предикат модифицируется как  $Curr \wedge (VC(a) == VC(b))$ , а в блок, соответствующий ложному значению,  $\neg Curr \wedge \neg(VC(a) == VC(b))$ .
- При слиянии потока управления предикатов  $Pred_1, Pred_2, \dots, Pred_n$  из базовых блоков-предшественников текущего предикат пути записывается как их дизъюнкция  $Corr = Pred_1 \vee Pred_2 \vee \dots \vee Pred_n$  и упрощается последовательным применением тождеств вида  $a \vee \neg a = true$ ,  $a \wedge \neg a =$

$$= false, a \vee a = a, a \wedge a = a.$$

Модель памяти программы, использующая ячейки памяти, остается неизменной. Для отслеживания неконстантных смещений дочерних ячеек памяти по отношению к родительским по-прежнему требуется определение целочисленных значений переменных, которое следует описанию раздела 2. Увеличивается точность отслеживания множеств *PtTo*: для каждой ячейки памяти  $ML_i$ , хранящейся во множестве, дополнительно отслеживается предикат  $Pred_i$ , который является истинным тогда, когда ячейка памяти этого множества указывает на данную ячейку  $ML_i$ . Предикаты состоят из выражений над классами значений и строятся непосредственно по следующему набору правил:

- При обработке присваивания и арифметических операций применяются обычные передаточные функции, а предикаты элементов множества не изменяются;
- При инициализации множества *PtTo* ячейкой памяти  $ML$  в ходе обработки оператора взятия адреса или при инициализации специальным значением *Undef* предикатом элемента становится текущий предикат пути  $Curr$ :  $\frac{a = \&b}{PtTo(ML(a)) = \{Curr: ML(b)\}}$ ;
- При условном переходе по сравнению двух указателей на неравенство применяются обычные передаточные функции: множества меняются только тогда, когда одно из них выражает must-отношение, предикаты элементов не меняются. При переходе по сравнению на равенство, как и ранее, строится пересечение множеств, и дополнительно предикаты одинаковых элементов объединяются через конъюнкцию:

$$\frac{if(p == q), PtTo(ML(p)) = \{Pred_i : ML_i\}, PtTo(ML(q)) = \{Pred_j : ML_j\}}{PtTo_{p==q}(ML(p)) = PtTo_{p==q}(ML(q)) = PtTo(p) \cap PtTo(q)},$$

$$\frac{ML_k \in PtTo_{p==q}, ML_k = ML_{i \in PtTo(ML(p))}, ML_k = ML_{j \in PtTo(ML(q))}}{Pred_k = Pred_i \wedge Pred_j}.$$

- При разыменовании указательной ячейки памяти и при слиянии потока управления множества  $PtTo$ , как и ранее, объединяются через функцию  $Unify$ . При этом нет необходимости в специальной обработке элемента  $Undef$ , т.к. теперь условие, при котором ячейка указывает на неизвестный элемент, отслеживается соответствующим предикатом:  $Unify(PtTo_a, PtTo_b) = PtTo_a \cup PtTo_b$ . При построении множеств  $PtTo$  необходимо обеспечить однократность вхождения в него каждой ячейки памяти, поэтому предикаты соответствующих ячеек объединяются дизъюнкцией.

$$\frac{a = *b, PtTo(b) = \{Pred_i : ML_i\}}{PtTo(ML(a)) = Unify_i(PtTo(ML_i))},$$

$$\frac{ML_k \in PtTo(ML_{i_1}), ML_k \in PtTo(ML_{i_2}), \dots, ML_k \in PtTo(ML_{i_n})}{Pred_k = \bigvee_{j=1}^n Pred_{i_j}}.$$

Дополнительно при обработке разыменования можно использовать следующее соображение: передавать функции  $Unify$  только те множества  $PtTo(ML)$ , для ячеек памяти которых хранящиеся предикаты  $Pred$  совместны с предикатом пути  $Curr$ . Тем самым варианты отношения указывания, которые не реализуемы на текущем пути выполнения, будут отсечены.

- При слиянии потока управления выполняется аналогичное объединение множеств  $PtTo$ , но к предикатам элементов множества дополнительно дописывается предикат пути блока, из которого пришло данное множество, а потом они объединяются дизъюнкцией:

$$\frac{n \geq 2, b_2, \dots, b_n \in Preds(b), PtTo_{b_i}(ML(a)) = \{Pred_j^{b_i} : ML_j^{b_i}\}}{PtTo_b(ML(a)) = Unify_i(PtTo_{b_i}(ML(a)))},$$

$$\frac{ML_k \in PtTo(ML_{i_1}), ML_k \in PtTo(ML_{i_2}), \dots, ML_k \in PtTo(ML_{i_n})}{Pred_k = \bigvee_{j=1}^n (Pred_{i_j} \wedge Curr_{b_j})}.$$

Предикаты, отслеживающие, из каких классов значений был сформирован данный класс при разыменовании и слиянии потока управления, создаются аналогичным образом. Сохраняются предикаты, истинность которых означает, что данный класс значения равен некоторому другому, т.е. хранится множество  $VCPreds(VC) = \{Pred_i : VC_i\}, \forall i, ji \neq j \models VC_i \neq VC_j$ . При разыменовании предикаты из множества  $PtTo$  дописываются через конъюнкцию к предикатам классов значений из  $VCPreds$ , а потом объединяются через дизъюнкцию. Аналогично происходит при слиянии потока управления, только используются предикаты пути из предшественников текущего базового блока. Сама передаточная функция, генерирующая новый класс значения, не меняется.

Соображение об отсечении несовместных конъюнкций из окончательной дизъюнктивной формулы может применяться и здесь. Напри-

мер, пусть обрабатывается инструкция  $k = *p$ , где  $PtTo(ML(p)) = \{vc_x > 0 : a, vc_x \leq 0 : b\}$ , при этом  $VCPreds(ML(a)) = \{vc_x > 5 : vc_1, vc_y = 2 : vc_2\}$ ,  $VCPreds(ML(b)) = \{vc_x = 7 : vc_1, vc_y > 5 : vc_2, vc_z = 0 : vc_3\}$ . Тогда обычная передаточная функция заведет новый класс значений  $vc_k$  для ячейки памяти переменной  $k$ , а предикаты для составных частей этого класса значений будут записаны как  $VCPreds(vc_k) = \{vc_x > 5 : vc_1, vc_x > 0 \wedge vc_y = 2 \vee vc_x \leq 0 \wedge vc_y > 5 : vc_2, vc_x \leq 0 \wedge vc_z = 0 : vc_3\}$ .

Для внутрипроцедурного анализа, создающего модель памяти, классы значений и их атрибуты, можно применять уже алгоритмы 2 и 4 соответственно. Действительно, передаточные функции, создающие множества  $PtTo$  и классы значений, остаются прежними за исключением того, что из множеств  $PtTo$  удаляются элементы с несовместными предикатами. Однако по

построению предиката пути по индукции легко видеть, что, во-первых, генерируемая логическая формула корректно отражает путь выполнения, соответствующий переходам по индивидуальным подформулам. Во-вторых, при выяснении совместности формулы все классы значений трактуются как свободные переменные — единственное ограничение, которое может потенциально нарушить корректность, это то, что все вхождения конкретного класса значений  $VC$  в формулу обозначают одно и то же значение. Но корректность выбора классов значений алгоритмом 4 гарантируется утверждением 4 (в заданных условиях просмотра ограниченного числа путей и отсутствия алиасов у параметров функции), а мы не меняем этот алгоритм — ведение предикатов состава данного класса значений не влияет на выбор. Тем самым построение множеств  $PtTo$  и классы значений является корректным, а корректность созданных предикатов следует из корректности построения предиката пути. **Доказано Утверждение 6.** Алгоритмы 2 и 4, вычисляющие модель памяти функции и классы значений при внутрипроцедурном анализе с описанными выше изменениями (обозначим их 2' и 4'), работают корректно в ограничениях утверждений 2 и 4, и предикаты для классов значений и вычисляются правильно.

Осталось обсудить изменения, которые происходят во время межпроцедурного анализа в части создания и использования аннотации функции. Алгоритм 5 выделения ячеек памяти и классов значений, которые попадут в аннотацию, не изменяется, однако дополнительно сохраняются предикаты, вычисленные для множеств  $PtTo$  и классов значений ( $VCPreds$ ). Кроме того, вводится ограничение  $ml$  на максимальное количество конъюнкций, которые содержатся в предикате. Если это количество превзойдено, то части “лишних” конъюнкций, которые были добавлены при обработке предиката пути, меняются на истину (другими словами, удаляются из формулы). При этом предикат становится более слабым, но сохраняет корректность: если предикат до удаления был истинен при некоторых значениях свободных переменных, то и после удаления он останется истинным. Будем обозначать этот алгоритм как алгоритм 5'.

Внешние классы значений используются по аналогии с утверждением 5, но передаточные функции множеств  $PtTo$  и других абстрактных значений теперь учитывают предикаты, как описано выше. При этом, т.к. исходные передаточные функции являются ассоциативными, и конъюнкция/дизъюнкция предикатов также ассоциативны, то требования утверждения 5 выполняются. Наконец, при использовании аннотации алгоритм сопоставления ячеек памяти и классов значений также остается неизменным.

#### 4.2. Чувствительность к путям на основе символьных выражений

Описанный выше анализ расширяет анализ предыдущего уровня на основе классов значений, которые остаются центральным понятием анализа, и условия того, что ячейка памяти содержит определенное значение, формулируются именно на основе классов значений. Однако с повышением уровня анализа можно заметить, что ценность классов значений как удобного способа отследить эквивалентность значений между переменными снижается. Во-первых, те же самые эквивалентности можно отследить и через запросы к SMT-решателю, если отразить все связи между классами значений через присваивания и выполнения операций в формуле запроса. Действительно, теория равенств и неинтерпретируемых функций (QF\_UF) в SMT-решателях может быть использована для проверки утверждений, которые делаются на основе классов значений: распространение одинаковых классов значений через присваивания аналогична аксиомам равенств, а назначение классов через поиск ранее записанных в хеш-таблице эквивалентностей по сути использует операции над классами значений как неинтерпретируемые функции. Тем самым, если SMT-решателю над теорией QF\_UF предъявить два выражения над классами значений, то он может ответить на вопрос, эквивалентны они или нет (т.е. являются ли они одним и тем же классом значений).

Во-вторых, при межпроцедурном анализе с использованием внешних значений уже делается шаг в сторону символьного выполнения, так как внешние значения при вычислениях внутрипроцедурного анализа трактуются как символьные переменные, а их конкретизация происходит при

применении аннотации вызываемой функции в вызывающей.

В-третьих, опыт реализации детекторов показывает, что, несмотря на то, что основные вычисляемые атрибуты целочисленных значений и множеств *PtTo* нужны большинству детекторов, эквивалентность значений через равенство классов значений может быть не такой, как требуется детектору. Другими словами, одинаковые классы значений могут иметь разные атрибуты в разных точках программы, хотя значения у них будут совпадать — различной будет информация, которая известна анализу об этом значении. Например, переходя по условию `if(a == 0)`, анализу становится известно, что класс значения, соответствующий ячейке памяти для переменной *a*, внутри этого условного оператора имеет нулевое значение. Однако сам класс значения не изменился, так как записи в переменную не было.

Из сказанного естественно вытекает идея организации чувствительного к путям анализа, отличная от подхода раздела 4.1. Откажемся от классов значений и будем ставить в соответствие ячейкам памяти напрямую символьные выражения *se*, которые в них хранятся. Такими выражениями будут константы, символьные переменные, одноместные и двуместные операции над символьными выражениями, которые поддерживаются в языке. Отслеживаются предикаты, при истинности которых символьное выражение равно некоторому другому (аналогично классам значений). Задачей анализа будет определение того, какие символьные выражения хранятся в ячейках памяти программы в каждой ее точке. Дополнительных атрибутов символьных выражений вычислять не будем — вместо этого каждый детектор должен будет строить собственную формулу выполнимости ошибочной ситуации с использованием предикатов над символьными выражениями. Совместность формулы будет проверяться с помощью SMT-решателей.

Для эффективного использования такого подхода необходимо также пересмотреть способ построения модели памяти. Если анализируется язык, допускающий адресную арифметику и произвольные смещения (неизвестные статически) дочерних ячеек памяти относительно

родительских, то полноценная поддержка символьной памяти будет требовательна к ресурсам. Поэтому целесообразно ограничить применение подхода данного раздела к языкам, в которых указателями нельзя управлять произвольно, например, к Java или C#. Тогда для обработки ситуации, в которой некоторая ячейка памяти может указывать на несколько других, будет достаточно вычислять множества *PtTo*, а интервалы целочисленных значений *Val* можно будет не поддерживать. Для массивов мы по-прежнему будем создавать не более чем *MaxSize* элементов массива и моделировать символьный доступ к массиву как последовательность не более чем *MaxSize* условных операторов (`ite`): `a[i]=i==0 ? a[0]:(i==1 ? a[1]: ... (i==MaxSize ? a[MaxSize]: a[undef])...)`. Внутри циклов можно отдельно использовать анализ индукционных переменных, показывающий возможные значения линейных выражений с участием этих переменных, для ограничения количества условных операторов.

Ограничим модельный язык рисунка 2, запретив арифметические операции с указателями и взятие адреса произвольной ячейки памяти. Указатель становится подобен ссылке на объект, которая может указывать на несколько объектов в зависимости от пути выполнения, но которую можно только присваивать и сравнивать на равенство, а также разыменовывать. Будем вычислять множества *PtTo* в чувствительном к путям выполнения варианте, отслеживая предикаты, при выполнении которых ячейка памяти указывает на ту или иную другую ячейку.

Передаточные функции, обновляющие символьные выражения для ячеек памяти, будут схожи с функциями для классов значений из раздела 3. Применение двуместных операций к символьным выражениям с предикатами приводит к попарному пересечению предикатов и применению операции к символьным выражениям-частям. При этом несовместные предикаты удаляются из результирующего множества, а предикаты одинаковых выражений объединяются через дизъюнкцию. Аналогично при разыменовании указательной ячейки памяти предикат, при истинности которого ячейка указывает на другую ячейку, присписывается к соответствующим предикатам

$$\begin{array}{c}
a = b \star c, \star \in \{+, -, \dots\} \\
\frac{a = \text{const}}{SE(ML(a)) = \text{const}} \quad \frac{a = b}{SE(ML(a)) = SE(ML(b))} \quad \frac{SE(ML(b)) = \{\langle Pred_i : SE_i \rangle\}, SE(ML(c)) = \{\langle Pred_j : SE_j \rangle\}}{SE(ML(a)) = \{\forall i, j \langle Pred_i \wedge Pred_j : SE_{ij} = SE_i \star SE_j \rangle\}} \\
\\
\frac{a = *b, PtTo(b) = \{Pred_i : ML_i\}, SE(ML_i) = \{\langle Pred_j^i : SE_j^i \rangle\}}{SE(ML(a)) = \{\forall i, j \langle Pred_{ij} = Pred_i \wedge Pred_j^i : SE_j^i \rangle\}} \\
\exists i, j, k, l : SE_j^i = SE_l^k \models Pred_{ij} = (Pred_i \wedge Pred_j^i) \vee (Pred_k \wedge Pred_l^k), SE(ML(a)) \setminus = \{SE_l^k\} \\
\\
\frac{\exists i, j : Pred_{ij} \models false}{SE(ML(a)) \setminus = \{SE_j^i\}} \\
\\
\frac{n \geq 2, b_1, b_2, \dots, b_n \in Pred(b), SE_{b_i}(ML(a)) = \{Pred_{b_i}^{b_i} : SE_{b_i}^{b_i}\}}{SE(ML(a)) = \begin{cases} \{\forall i, j \langle (Pred_i \wedge Curr_{b_1}) \vee (Pred_j \wedge Curr_{b_2}) : SE_{ij} = Unify(SE_{b_1}(ML(a)), SE_{b_2}(ML(a))) \rangle\}, n = 2 \\ Unify(Unify_{b_i}^{i=1, n-1}(SE_{b_i}(ML(a))), SE_{b_n}(ML(a))), n > 2 \end{cases}}
\end{array}$$

Рис. 3.

частей символьного выражения, результирующее множество частей символьных выражений и их предикатов упрощается так же. При слиянии потока управления приписываются соответствующие предикаты путей для базовых блоков-предшественников текущего блока (см. рис. 3).

Алгоритм выполнения внутрипроцедурного и межпроцедурного анализа строится аналогично алгоритму 4'. При обходе графа потока управления обновляются ячейки памяти, множества *PtTo* и символьные выражения, хранящиеся в ячейках, согласно описанным выше передаточным функциям. Строится предикат пути, который необходим для корректного вычисления символьных выражений и детекторов. По-прежнему выполняется анализ ограниченного количества итераций цикла. Распиряющий оператор  $\nabla$  не применяется.

При межпроцедурном анализе алгоритм построения аннотации функции остается неизменным — используется алгоритм 5', но вместо классов значений вместе с ячейками памяти сохраняются их символьные выражения *se*. Когда аннотация используется, то нет необходимости отдельно трактовать символьно внешние ячейки памяти и выражения, хранящиеся в них, т.к. все вычисления уже выполняются символьно. Если для ячеек памяти, соответствующих формальным параметрам, известно, что в них в вызывающей функции содержится символьное выражение  $SE(ML(param)) = \{\langle Pred_i : SE_i \rangle\}$ ,

то во всех символьных выражениях, где используется некоторая символьная переменная формального параметра *se<sub>param</sub>* с предикатом *Pred*, подставляется и раскрывается выражение  $\{\langle Pred \wedge Pred_i : SE_i \rangle\}$ , т.е. во всех последующих вхождениях выбрасываются несовместные предикаты, а предикаты для одинаковых выражений объединяются через дизъюнкцию. Эта процедура выполняется для всех фактических параметров одновременно, чтобы учесть возможные зависимости между ними.

## 5. ЗАКЛЮЧЕНИЕ

Описанные выше алгоритмы многоуровневого анализа были реализованы в течение ряда лет в инструменте Svace вместе со значительным набором детекторов. Для первого уровня анализа детекторы реализованы в инфраструктуре JavaC, Clang Static Analyzer и Roslyn для Java, Си/Си++ и C#, соответственно. Внутрипроцедурный анализ потока данных реализован в общем движке Svace для Java и Си/Си++, так как он не только находит ошибки, но и готовит данные для последующих уровней. Анализ C# полностью реализован в собственном движке на основе инфраструктуры Roslyn [8].

Всего в анализаторе Svace около 450 детекторов, кроме того, еще около 340 детекторов — это детекторы первого уровня из сторонних инструментов. Около 300 детекторов реализованы в основном движке Svace (это детекторы второго и

третьего уровня анализа), подавляющее большинство из них работают для Си, Си++, Java. Примерно 20 детекторов первого уровня реализованы нами в Clang Static Analyzer, 11 детекторов — в компиляторе Javac. 35 детекторов реализованы для языка C#.

Время анализа больших проектов, которые постоянно проверяются Svasc, составляет около 5 часов: Android 5 — 4 ч 55 мин, Tizen 2.3 — около 3 часов, ядро Linux 3.17 — около часа. Качество анализа проверяется собственными тестами (более 2000) и постоянной оценкой больших проектов. В зависимости от детектора оно составляет 60–90% истинных срабатываний, причем для Java этот показатель обычно больше, чем для Си++, из-за простоты модели памяти языка.

В ходе внедрения инструмента Svasc разработанные алгоритмы анализа показали свою состоятельность. Хотелось бы отметить, что выполнения всех упомянутых во введении требований к анализатору можно достичь только упорной многолетней работой над результатами использования в промышленном окружении.

## СПИСОК ЛИТЕРАТУРЫ

1. *Кнут Д.* Искусство программирования. Том 3. Сортировка и поиск. Вильямс, 2014.
2. *Strein D., Kratz H., Lowe W.* Cross-Language Program Analysis and Refactoring. Proceedings of the Sixth IEEE International Workshop on Source Code Analysis and Manipulation (SCAM'06). IEEE Computer Society, Washington, DC, USA, pp. 207–216.
3. *Xu Z., Kremenek T., Zhang J.* A memory model for static analysis of C programs. Proceedings of the 4th international conference on Leveraging applications of formal methods, verification, and validation, Volume Part I (ISoLA'10), Tiziana Margaria and Bernhard Steffen (Eds.), Part I. Springer-Verlag, Berlin, Heidelberg, 2010, pp. 535–548.
4. *Cousot P., Cousot R.* Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages (POPL'77). ACM, New York, NY, USA, pp. 238–252.
5. *Shengyue Wang, Xiaoru Dai, Kiran S. Yellajosula, Antonia Zhai, and Pen-Chung Yew.* 2005. Loop selection for thread-level speculation. In Proceedings of the 18th international conference on Languages and Compilers for Parallel Computing (LCPC'05), Eduard Ayguade, Gerald Baumgartner, J. Ramanujam, and P. Sadayappan (Eds.). Springer-Verlag, Berlin, Heidelberg, 2005, pp. 289–303.
6. *Бородин А.Е.* Межпроцедурный контекстно-чувствительный статический анализ для поиска ошибок в исходном коде программ на языках Си и Си++: дис. ... канд. ф.-м. наук. Москва, 2016 г.
7. Tucker Taft. The use of value numbers in static analysis.  
<http://www.adacore.com/knowledge/technical-papers/the-use-of-value-numbers-in-static-analysis/>
8. *Кошелев В.К., Игнатъев В.Н., Борзилов А.И.* Инфраструктура статического анализа программ на языке C# // Труды Института системного программирования РАН. 2016. Т. 28. № 1. С. 21–40.
9. *Иванников В.П., Белеванцев А.А., Бородин А.Е., Игнатъев В.Н., Журихин Д.М., Аветисян А.И., Леонов М.И.* Статический анализатор Svasc для поиска дефектов в исходном коде программ // Труды Института системного программирования РАН. 2014. Т. 26. С. 231–250.
10. *Бородин А.Е., Белеванцев А.А.* Статический анализатор Svasc как коллекция анализаторов разных уровней сложности // Труды Института системного программирования РАН. 2015. Т. 27. № 6. С. 111–134.